


# Reescrita de Sistema Legado — ConectaAtende API (.NET 8)

## PARTE 1 — CENÁRIO, CONTEXTO, PROBLEMA E MOTIVAÇÕES

---

### 1. Contextualização Organizacional

A ConectaAtende S.A. é uma empresa brasileira que atua no segmento de soluções digitais para pequenas e médias empresas. Seu principal produto é um sistema de:

- Agenda corporativa;
- Central de atendimento (tickets);
- Organização básica de contatos e relacionamento com clientes.

O sistema está em operação há vários anos e possui clientes ativos que dependem dele diariamente para realizar atendimentos, registrar solicitações e organizar sua base de contatos.

Apesar de cumprir seu papel funcional, o sistema atual apresenta sinais claros de desgaste arquitetural e técnico.

---

## 2. Situação Atual (Sistema Legado)

O produto da ConectaAtende foi desenvolvido ao longo do tempo sem um modelo arquitetural bem definido. À medida que novas funcionalidades foram sendo incorporadas, decisões pontuais passaram a comprometer a estrutura geral do sistema.

Atualmente, o legado apresenta:

- Forte acoplamento entre módulos;
- Código difícil de testar;
- Regras de negócio misturadas com infraestrutura;
- Dificuldade de manutenção;
- Lentidão perceptível em operações críticas;
- Ausência de mecanismos de reversão de ações;
- Estrutura pouco preparada para integração com outros sistemas.

Além disso, o crescimento da base de clientes aumentou o volume de dados manipulados, tornando problemas de organização interna mais evidentes.

---

## 3. Problemas Identificados

A análise técnica do sistema atual identificou os seguintes pontos críticos:

### 3.1 Catálogo de Contatos

- Busca por nome ou telefone torna-se lenta conforme o volume cresce.
- Atualizações de contato não refletem corretamente nas consultas.
- Telefones duplicados geram inconsistências.
- Exclusões deixam dados órfãos em estruturas internas.

### 3.2 Central de Atendimento

- O mecanismo de “próximo atendimento” não é flexível.
- A troca de regra de triagem exige alteração direta de código.

- O comportamento não é previsível em horários de pico.
- Não há clareza sobre como os tickets são organizados internamente.

**3.3 Experiência do Usuário**

- Não há funcionalidade consistente de “últimos acessos”.
- Exclusões e alterações não podem ser revertidas.
- O suporte técnico tem dificuldade de recuperar dados.

---

## 4. Direcionamento Estratégico da Empresa

A ConectaAtende decidiu iniciar uma reestruturação técnica gradual, começando pela reescrita do núcleo funcional do sistema.

O objetivo não é apenas modernizar a tecnologia, mas:

- Corrigir problemas estruturais;
- Garantir previsibilidade de comportamento;
- Organizar corretamente regras de negócio;
- Preparar a solução para integração com novos canais digitais;
- Construir uma base técnica sustentável.

---

## 5. Escopo da Primeira Fase da Reescrita

Nesta etapa inicial, será desenvolvida uma API em .NET 8 com persistência InMemory, contemplando:

1. Catálogo de contatos;
2. Central de atendimento (tickets);
3. Política de triagem configurável;
4. Mecanismo de reversão de operações;
5. Lista de contatos recentes;
6. Projeto de benchmark para análise comparativa.

A escolha por persistência em memória é estratégica:

- Reduz complexidade inicial;
- Permite foco em modelagem e estrutura;
- Facilita testes e experimentação;
- Prepara o sistema para futura substituição por banco de dados real.

---

## 6. Objetivos Técnicos da Nova Solução

A nova API deverá:

- Separar responsabilidades claramente;
- Isolar regras de negócio da infraestrutura;
- Permitir troca de implementação sem quebrar contratos;
- Sustentar volume de dados simulado;
- Oferecer comportamento consistente após atualizações;
- Permitir medição de cenários críticos.

Além disso, a API será projetada para:

- Comunicação com aplicações web;
- Integração com serviços externos;
- Evolução futura para consumo por aplicações mobile.

---

## 7. Motivação Acadêmica da Atividade

Este trabalho simula uma situação real de mercado: a reescrita de um sistema legado.

O objetivo pedagógico é permitir que o aluno:

- Compreenda os impactos de decisões estruturais;
- Relacione estruturas de dados ao comportamento do sistema;
- Trabalhe com separação de camadas;
- Analise trade-offs técnicos;

- Justifique decisões com base em evidência;
- Desenvolva pensamento arquitetural.

O foco não é apenas implementar funcionalidades, mas:

- Construir uma solução coerente;
- Manter consistência interna;
- Pensar em escalabilidade conceitual;
- Aplicar fundamentos de engenharia de software.

---

## 8. Papel do Aluno no Projeto

O aluno assume o papel de engenheiro responsável pela reescrita do núcleo do sistema.

Ele deverá:

- Modelar corretamente o domínio;
- Escolher abordagens adequadas;
- Garantir consistência dos dados;
- Organizar a arquitetura;
- Medir cenários relevantes;
- Documentar decisões.

A qualidade da justificativa técnica será tão importante quanto o funcionamento do sistema.

---

### 9. Resultado Esperado desta Fase

Ao final desta etapa, espera-se que o aluno tenha:

- Construído uma API organizada e funcional;
- Aplicado conceitos de arquitetura limpa;
- Tomado decisões estruturais conscientes;
- Produzido documentação técnica clara;

- Demonstrado capacidade analítica ao comparar cenários.

# PARTE 2 — TIPO DE APLICAÇÃO, ESCOPO TÉCNICO E MODELO ARQUITETURAL

---

## 1. Tipo de Aplicação a Ser Desenvolvida

Nesta fase da reescrita, será desenvolvida uma API REST em .NET 8, responsável por fornecer os serviços centrais da plataforma ConectaAtende.

A API será:

- Stateless;
- Orientada a contratos HTTP;
- Preparada para consumo por aplicações web;
- Projetada para futura integração com aplicativos mobile;
- Estruturada para integração com serviços externos.

A aplicação deverá possuir organização arquitetural adequada, permitindo evolução futura sem reestruturação drástica.

---

## 2. Escopo Funcional da API

A API deverá contemplar os seguintes módulos obrigatórios:

### 2.1 Catálogo de Contatos

Responsável por:

- Cadastro de contatos;
- Atualização de informações;
- Exclusão controlada;
- Busca por identificador;
- Busca por nome;

- Busca por telefone;
- Listagem paginada.

---

### 2.2 Central de Atendimento (Tickets)

Responsável por:

- Registro de ticket associado a contato;
- Enfileiramento para atendimento;
- Consulta do próximo ticket;
- Retirada para atendimento;
- Atualização de status.

---

### 2.3 Política de Triagem Variável

A API deverá permitir alternância dinâmica entre diferentes políticas de atendimento:

- Ordem de chegada;
- Prioridade;
- Política mista (critério composto).

A mudança de política não poderá comprometer a consistência do sistema.

---

### 2.4 Mecanismo de Reversão (Undo)

Deverá existir mecanismo para desfazer a última operação válida relacionada a contatos.

O sistema deve manter coerência após a reversão.

---

### 2.5 Lista de Contatos Recentes

Ao acessar um contato, ele deverá ser registrado como “recente”.

A lista deverá:

- Ter capacidade limitada;
- Atualizar posição quando houver novo acesso;
- Permanecer consistente após exclusões.

---

### 2.6 Projeto de Benchmark

Deverá ser criado projeto independente para medição de cenários críticos.

Os benchmarks deverão analisar:

- Inserção de grande volume de dados;
- Consultas frequentes;
- Atualizações;
- Impacto estrutural nas operações.

Os resultados deverão ser documentados e interpretados.

---

### 2.7 Estrutura Associativa Própria (Desafio Avançado)

Como atividade complementar, deverá ser implementada uma estrutura associativa didática com:

- **Buckets** (Estrutura interna (normalmente um vetor) onde as chaves são distribuídas com base em uma função de dispersão; serão utilizados para organizar os dados da estrutura associativa didática.)
- **Tratamento** de **colisões** (Mecanismo para lidar com situações em que duas ou mais chaves mapeiam para o mesmo bucket; será utilizado para garantir que múltiplos elementos possam coexistir corretamente na mesma posição lógica.)
- **Operações básicas** de inserção, busca e remoção.

Deverá ser feita comparação conceitual com a alternativa padrão utilizada no framework.

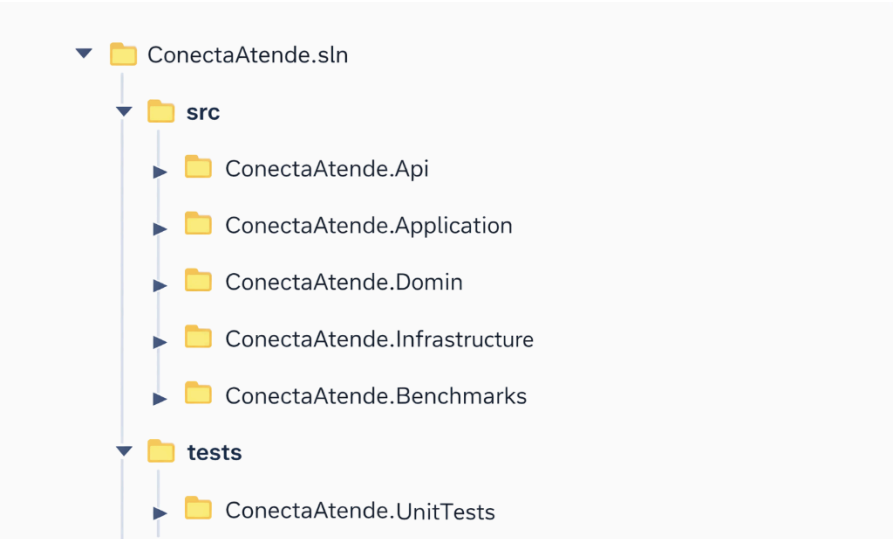
---

## 3. Modelo Arquitetural Esperado

A aplicação deverá seguir princípios de Arquitetura Limpa, garantindo separação clara entre domínio, aplicação e infraestrutura.

---

### 3.1 Estrutura da Solution




---

### 3.2 Responsabilidades por Camada

3.2.1 Camada Domain

Responsável por:

- Entidades;
- Objetos de valor;
- Regras de negócio puras;
- Interfaces de repositório.

Não deve depender de infraestrutura ou frameworks.

---

3.2.2 Camada Application

Responsável por:

- Casos de uso;
- Orquestração das regras;

- Serviços de aplicação;
- DTOs.

Deve depender apenas do domínio.

---

3.2.3 Camada Infrastructure

Responsável por:

- Implementações InMemory;
- Estruturas internas utilizadas;
- Implementação das políticas de triagem;
- Mecanismo de undo;
- Lista de recentes.

Depende do domínio e da aplicação.

---

3.2.4 Camada API

Responsável por:

- Controllers;
- Configuração de injeção de dependência;
- Exposição de endpoints;
- Swagger.

Não deve conter regra de negócio.

---

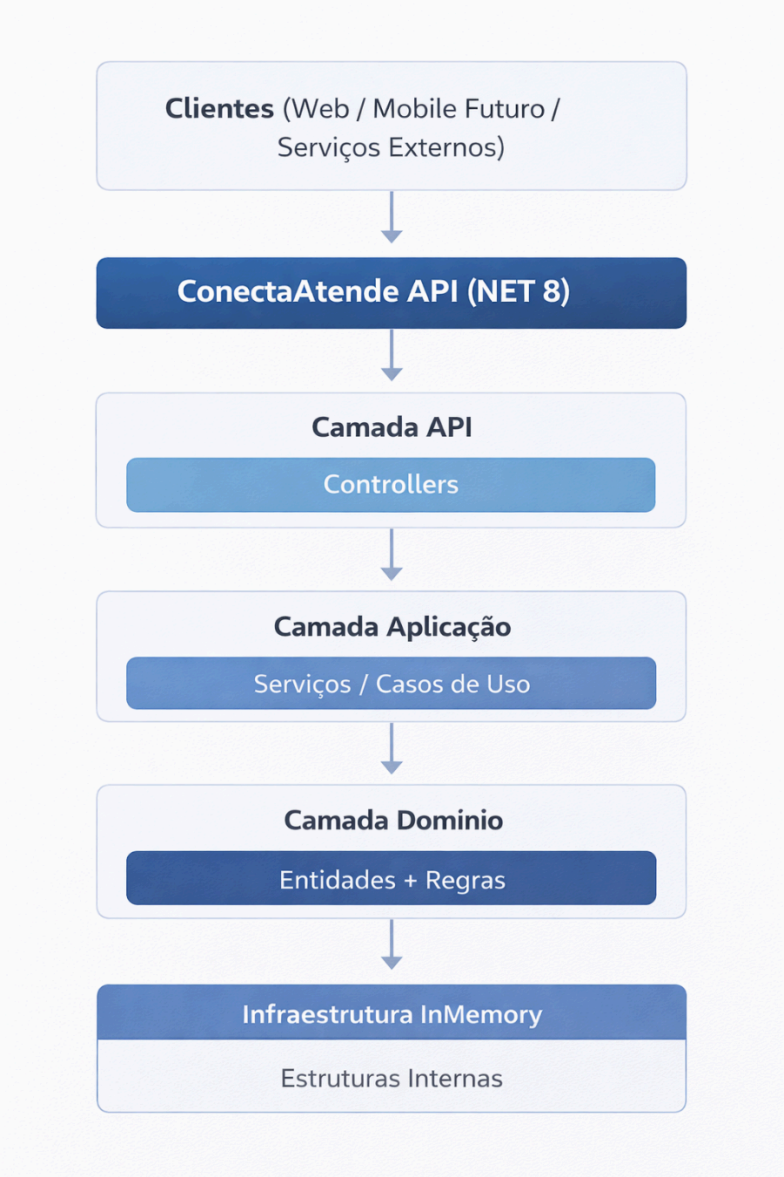
3.2.5 Projeto de Benchmarks

Responsável por:

- Medição isolada de cenários;
- Comparação entre abordagens;
- Produção de evidências quantitativas.

## 4. Diagrama Conceitual da Arquitetura

A arquitetura deverá respeitar o seguinte fluxo conceitual:




## 5. Objetivos Técnicos da Estrutura Proposta

A organização proposta busca:

- Reduzir acoplamento;

- Aumentar testabilidade;
- Permitir substituição futura de persistência;
- Isolar decisões estruturais;
- Facilitar evolução incremental.

---

## 6. Diretrizes de Qualidade

A aplicação deverá:

- Compilar sem warnings críticos;
- Apresentar código legível;
- Seguir boas práticas de organização;
- Possuir nomenclatura clara;
- Evitar duplicação desnecessária de lógica;
- Conter documentação mínima no README.

---

## 7. Resultado Esperado da Parte 2

Ao concluir esta etapa, espera-se que o aluno tenha:

- Uma API estruturada corretamente;
- Separação clara entre camadas;
- Implementação funcional dos módulos;
- Base preparada para medição;
- Projeto organizado conforme arquitetura proposta.

# PARTE 3 — EXERCÍCIOS, FORMA DE ENTREGA E ACORDOS DE TRABALHO

---

## 1. Exercícios Obrigatórios

A seguir, estão definidos os módulos que compõem a implementação obrigatória da atividade.
Todos os módulos deverão ser expostos por meio da ConectaAtende API (.NET 8), seguindo o modelo arquitetural estabelecido na Parte 2.

A persistência deverá ocorrer exclusivamente em memória.

---

### 1.1 Catálogo de Contatos

Deverá ser implementado o módulo responsável pela manutenção da base de contatos da aplicação.

Endpoints esperados (mínimo)

- POST /contacts
- GET /contacts/{id}
- PUT /contacts/{id}
- DELETE /contacts/{id}
- GET /contacts?page=&pageSize=

Camadas envolvidas

- Domain: entidade Contact, regras básicas.
- Application: ContactService.
- Infrastructure: repositório InMemory.
- API: Controller de contatos.

Funcionalidades obrigatórias

- Cadastro de novos contatos;
    - Cadastro pode ter mais de um contato 1-N;
- Atualização de informações;
- Exclusão consistente;
- Busca por identificador;

- Busca por nome podendo;
    - Retornar lista de busca parcial (mínimo 3 caracteres);
    - Ignorar acentuação e case sensitive;
- Busca por telefone;
- Listagem paginada.

Requisitos essenciais

- Nome obrigatório;
- Telefones normalizados (remoção de caracteres não numéricos);
- Prevenção de inconsistências internas;
- Atualizações refletidas corretamente nas consultas;
- Exclusão deve impactar imediatamente as buscas.

A implementação deve garantir integridade dos dados mesmo após múltiplas operações consecutivas.

---

**1.2 Busca Otimizada**

A aplicação deverá apresentar comportamento consistente quando submetida a volume simulado de dados.

Requisitos técnicos

- Implementar mecanismo de geração de dados (ex.: endpoint /dev/seed em ambiente Development).
- Permitir geração parametrizada de contatos.
- Garantir que a busca continue funcional após atualizações e exclusões.

Endpoints sugeridos

- GET /contacts/search?name=
- GET /contacts/search?phone=

Expectativa

O aluno deverá justificar no README:

- Como estruturou a busca internamente;
- Quais compromissos assumiu;
- Como garantiu consistência após alterações.

---

**1.3 Central de Atendimento (Tickets)**

Deverá ser implementado o módulo de tickets associado a contatos existentes.

Endpoints esperados

- POST /tickets
- POST /tickets/enqueue/{ticketId}
- GET /tickets/next
- POST /tickets/dequeue

Regras obrigatórias

- Ticket só pode ser criado para contato válido;
- Ticket não pode ser enfileirado duas vezes;
- Ao retirar da fila, o status deve ser alterado;
- Sistema deve manter coerência de estados.

Camadas envolvidas

- Domain: entidade Ticket.
- Application: TicketService.
- Infrastructure: estrutura interna para controle de fila.
- API: Controller de tickets.

A implementação deve impedir inconsistências após múltiplas operações.

---

**1.4 Política de Triagem**

A aplicação deverá permitir alternância entre políticas de atendimento.

Endpoints esperados

- POST /triage/policy
- GET /triage/policy

Políticas mínimas

- Ordem de chegada;
- Prioridade;
- Política mista.

Requisitos

- Mudança de política não pode invalidar dados existentes;
- Deve ser possível alterar política durante execução;
- Implementação deve respeitar separação de responsabilidades.

A lógica de escolha do próximo ticket não deve estar no controller.

---

**1.5 Mecanismo de Undo**

Deverá ser implementado mecanismo para desfazer a última operação válida relacionada ao módulo de contatos.

Endpoint esperado

- POST /contacts/undo

Requisitos

- Deve reverter última atualização ou exclusão;
- Sistema deve manter coerência após reversão;
- Caso não exista operação a desfazer, retornar resposta apropriada;
- Não deve comprometer consistência das buscas.

A responsabilidade da pilha de operações não deve estar na camada de API.

---

**1.6 Lista de Contatos Recentes**

A aplicação deverá manter lista de contatos recentemente acessados.

Comportamento esperado

- Ao executar GET /contacts/{id}, atualizar lista de recentes;
- Lista deve possuir capacidade máxima configurável;
- Contato acessado novamente deve assumir posição mais recente;
- Exclusão deve remover contato da lista de recentes.

Endpoint sugerido

- GET /contacts/recent?limit=

Expectativa técnica

A implementação deve:

- Manter consistência entre catálogo e lista de recentes;
- Evitar crescimento ilimitado;
- Manter desempenho adequado sob acessos repetidos.

---

**1.7 Projeto de Benchmark**

Deverá ser criado projeto separado (ConectaAtende.Benchmarks) utilizando BenchmarkDotNet.

Cenários mínimos a serem medidos

- Inserção de grande volume de contatos;
- Busca por nome;
- Busca por telefone;
- Atualização de contatos em volume significativo.

Requisitos

- Rodar em modo Release;
- Documentar resultados no README;
- Apresentar interpretação técnica dos dados;
- Comparar cenários distintos quando aplicável.

A análise deve evidenciar entendimento do impacto estrutural das decisões.

---

### 1.8 Estrutura Associativa Didática (Atividade Complementar)

Como atividade avançada, deverá ser implementada estrutura associativa própria.

Componentes obrigatórios

- Buckets (estrutura base de armazenamento);
- Tratamento de colisões;
- Operações básicas:
    - Inserção;
    - Busca;
    - Remoção.

---

## 2. Estrutura Geral da Atividade

Esta etapa corresponde à implementação prática da reescrita do núcleo do sistema ConectaAtende, conforme descrito nas Partes 1 e 2.

O aluno deverá desenvolver uma API funcional, organizada segundo o modelo arquitetural proposto, contemplando os módulos obrigatórios e evidenciando decisões técnicas fundamentadas.

O trabalho poderá ser realizado:

- Individualmente;

Ou em grupo de até 3 integrantes.

### 2.1 Forma de Entrega

A entrega deverá ocorrer exclusivamente por meio de repositório Git.

O repositório deverá conter:

- Solution organizada conforme estrutura proposta;
- Código compilável;
- API executável;
- Projeto de benchmark funcional;
- README obrigatório.

---

## 3. Conteúdo Obrigatório do README

O README deverá conter:

1. Descrição da arquitetura adotada;
2. Organização da solution;
3. Justificativa das decisões técnicas;
4. Explicação dos trade-offs assumidos;
5. Resultados e interpretação dos benchmarks;
6. Instruções para execução da API e dos benchmarks;
7. Seção “Declaração do Grupo”.

---

## 4. Acordos de Trabalho

Para garantir uniformidade e qualidade técnica, estabelecem-se os seguintes acordos:

- Persistência exclusivamente InMemory;
- Separação de camadas conforme modelo arquitetural proposto;
- Controllers não devem conter regra de negócio;
- O domínio não deve depender da infraestrutura;
- Código deve compilar sem erros;
- Organização clara de namespaces;
- Nomenclatura coerente;
- Evitar duplicação de lógica.

---

## 5. Diferencial (Plus)

Será considerado diferencial:

- Testes unitários com cobertura mínima de 70%;

- Evidência da cobertura apresentada no repositório;
- Organização de testes por camada;
- Clareza na validação de regras críticas.

---

## 6. Critérios de Avaliação

A avaliação considerará:

- Aderência ao domínio;
- Organização arquitetural;
- Coerência das decisões técnicas;
- Evidência e interpretação de benchmarks;
- Qualidade e clareza do código;
- Conformidade com os acordos de trabalho;
- Testes (quando aplicável).

---

## 7. Considerações Finais

Este trabalho representa a simulação de um cenário real de mercado: a reescrita de um sistema legado.

Espera-se que o aluno demonstre:

- Capacidade de modelagem;
- Consciência arquitetural;
- Raciocínio técnico;
- Organização estrutural;
- Análise crítica.

**A qualidade da justificativa técnica será considerada tão relevante quanto a implementação funcional**

*“O uso de IA é permitido, desde que o aluno consiga explicar integralmente cada decisão implementada.”*